Virtual KAM – submission note

Safal Gupta · Snapdeal AI First Intern assignment

What I built

A browser voice agent, "Asha", that acts as a Key Account Manager for a Snapdeal seller. The brief asks for a bot that goes beyond answering questions, so I designed it around four things an account owner does: know the account before the call, find the real cause, act during the call, and come back to check.

How one call works

1. The seller clicks Call (or Asha calls first, when the nightly scan flags the account).
2. Before speaking, the agent runs the **diagnosis engine** on the seller's data.
3. Asha states what she found, then asks only what the data cannot show (for example, *why* the products ran out).
4. She proposes two or three actions. When the seller says yes, a tool saves the action and it appears on screen.
5. She books a follow-up, reads a short summary, and the call is logged for the team dashboard.
6. On the next call she opens with what was agreed and what has happened since.
7. A person on the team can open any call, read it in full, and sign it off or flag it.

What the brief's data says (computed by code, not the LLM)

Sales and orders both fell 32.5% while average price stayed at ₹500, so it is a volume problem. With my mock product data, stockouts explain ₹45,450 (70%) of the ₹65,000 drop; the remaining ₹19,550 is attributed to lower visibility linked to cancellations. Cancelled orders are worth ₹18,900 this month, ₹8,100 more than at last month's rate.

AI models and voice technology

Layer	Choice	Why
Audio transport and agent framework	LiveKit Agents (open source, WebRTC)	Real-time, interruptible, one room per call, scales by adding workers
Speech-to-text	Sarvam Saaras v4 (code-mix mode for Hinglish)	Built for Indian accents and mixed Hindi-English
LLM	Sarvam 105B with function calling, reasoning off	Handles conversation and chooses tools; one vendor, one key
Text-to-speech	Sarvam Bulbul (v4 Flash, v3 as fallback)	Indian voices
Premium option	Gemini Live speech-to-speech	Selectable per call on the call screen; falls back to Sarvam if unavailable
Numbers and decisions	Plain Python diagnosis engine	The LLM never calculates; every figure is traceable and tested
Data	Postgres	Calls, actions, follow-ups, memory
Web	Next.js	Call screen, own-numbers mode, team dashboard

Design decisions I can defend

- **The LLM talks; code computes.** This removes invented numbers and keeps the LLM small and cheap.
- **Actions need a spoken yes, checked in code.** Code reads the seller's own transcribed words and saves an action only if they clearly agreed to that specific action (English, Hindi, Hinglish). I added this after a test call where the model acted without consent. A later Hindi call showed the check was too strict (it refused a plain "kar dijiye" because of the Hindi full-stop mark), so that became a test as well.
- **Every call is checked afterwards.** Each rupee amount the agent spoke is compared with the data; anything unmatched is flagged for the human reviewer. This came from a call where the model said "6 lakh" for 2 lakh.
- **The seller can undo.** "I did not agree to that" removes the action and reverses it.
- **Three sellers, three causes** (stockouts, rival pricing, late dispatch), plus a form to enter any numbers, to show the reasoning is not scripted.
- **State lives in the database, not the agent**, so calls are independent and workers can scale.
- **Built to survive bad networks.** Transcription and voice each have a fallback path, and a dropped call is recorded and resumed on the next one.
- **Tested without a microphone.** 117 automated tests, including the real agent run in a text-only session with a scripted LLM.

Cost and scale (estimates from published prices, not load-tested)

About ₹1.4–1.8 per call minute in the default mode, depending on the voice model. At 1 lakh sellers and 8 call minutes each per month, that is roughly ₹11–14 lakh a month, or ₹11–14 per seller. The nightly scan is one batch job with no LLM cost.

Assumptions and limits

- Sales in the brief are treated as before cancellations (sales = orders × average price).
- Product-level data, visits, rating, cancellation reasons and sellers S2/S3 are mock data I added.
- Actions are saved records with mock effects; nothing touches a real Snapdeal system.
- Not load-tested. Recovery figures are upper bounds ("up to").